МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3 ВАРИАНТ 2

по дисциплине «Построение и анализ алгоритмов»

Тема: Редакционное расстояние

Студентка гр. 4345 _____ Левкович Ю.С.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2025

Цель работы.

Изучить и реализовать алгоритм Вагнера-Фишера для поиска редакционного расстояния при разной стоимости операций, а также при особенной стоимости для специальных символов.

Задание

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. replace*(ε, a, b)* – заменить символ a на символ b.
2. insert*(ε, a)* – вставить в строку символ a (на любую позицию).
3. delete*(ε, b)* – удалить из строки символ b.

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки A и B, а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A в строку B.

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A; третья строка – B.

Выходные данные: одно число – минимальная стоимость операций.

Sample Input:

1 1 1

entrance

reenterable

Sample Output:

5

Для всех вариантов:

1) Предполагается, что для весов операций действует правило треугольника: если две последовательные операции можно заменить одной, то это не ухудшает общую цену.

2) При выполнении операций запрещается применять операции к символам, которые уже были получены в результате выполнения операций (т.е. строка преобразуется всегда только слева направо) — актуально для вариантов 3–6, 15 (упрощающее условие).

Вариант 2.

"Особый заменитель и особо удаляемый символ": цена замены на определённый символ отличается от обычной цены замены; цена удаления другого (или того же) определённого символа отличается от обычной цены удаления. Особый заменитель и цена замены на него, особо удаляемый символ и цена его удаления — дополнительные входные данные.

Выполнение работы:

В ходе выполнения работы была разработана функция vagner_fisher_alg – она представляет собой основной цикл работы алгоритма. Функция принимает на вход исходную и итоговую строки, стоимости каждых операций и специальные стоимости и символы.

Первым делом создаётся матрица размера N + 1, M + 1, по умолчанию она инициализируется нулями, а далее будет заполняться с помощью рекуррентной формулы, на которой и основан алгоритм:

$$Table(i,j) = \begin{cases} 0, i = 0, j = 0 \\ \mathrm{i} \cdot \mathrm{del_cost}, \mathrm{j} = 0, \mathrm{i} > 0 \\ \mathrm{j} \cdot \mathrm{ins_cost}, \mathrm{j} = 0, \mathrm{j} > 0 \\ \min = \begin{cases} \mathrm{Table(i, j - 1) + ins_cost} \\ \mathrm{Table(i - 1, j) + del_cost} \\ \mathrm{Table(i - 1, j - 1) + rep_cost(S1[i], S2[j])} \end{cases}, \mathrm{j} > 0, \mathrm{i} > 0 \end{cases}$$

Первая строка отвечает за стоимости операции удаления символов из исходного слова, а первый столбец за их добавления. Далее просматриваются незаполненные клетки таблицы из верхнего левого угла до правого нижнего. В каждой ячейке просматриваются три соседа, которые уже посчитаны, и к ним прибавляются стоимости соответствующих операций (путь сверху вниз указывает на удаление символа, путь слева вправо на добавление, а путь по

диагонали на замену (по сути, комбинация удаления и добавления, но из-за стоимости операций может выйти дешевле чем эти последовательные действия или наоборот дороже)). Результатом работы алгоритма будет значение в правой нижней ячейке, так как именно на этом шаге совпадают последние символы (значит прошлые уже совпали).

Оценка временной сложности алгоритма показывает, что она равна O(N*M), так как в процессе программа проходит по каждой ячейке двумерного массива размера N на M.

Оценка сложности работы алгоритма по памяти показывает, что для хранения таблицы пространственная сложность также будет равна O(N*M).

Исходный код программы смотри в Приложении 1.

№	Входные данные	Выходные данные	Таблица	Комментарий
1	1 1 1 cat parrot a a 1 1	4	p a r r o t. [0, 1, 2, 3, 4, 5, 6] c [1, 1, 2, 3, 4, 5, 6] a [2, 2, 1, 2, 3, 4, 5] t [3, 3, 2, 2, 3, 4, 4]	Работа алгоритма для простого преобразование одного слова в другое, все стандартные стоимости равны, нет специальных стоимостей операций.
2	4 1 2 cat parrot a a 4 2	6	p a r r o t. [0, 1, 2, 3, 4, 5, 6] c [2, 3, 4, 5, 6, 7, 8] a [4, 5, 3, 4, 5, 6, 7] t [6, 7, 5, 6, 7, 8, 6]	Работа алгоритма преобразования одного слова в другое, все стандартные стоимости разные, нет специальных сложностей операций.
3	1 1 1 cat parrot r a 3 4	4	p a r r o t. [0, 1, 2, 3, 4, 5, 6] c [1, 1, 2, 3, 4, 5, 6] a [5, 2, 1, 2, 3, 4, 5] t [6, 3, 2, 3, 4, 4, 4]	Работа алгоритма преобразования одного слова в другое, все стандартные стоимости одинаковые, есть специальные сложности операций.
4	1 1 1 cat r a	6	[0] c [1] a [5] t [6]	Работа алгоритма преобразования одного слова в пустое слово, вычисляется стоимость удаления всех символов, включая специальные.

	3 4			
5	1 1 1 parrot r a 3 4	6	p a r r o t . [0, 1, 2, 3, 4, 5, 6]	Работа алгоритма преобразования пустого слова в не пустое слово, вычисляется стоимость добавления всех символов.

Таблица 1 – тестирование работы программы.

Вывод.

Был изучен и реализован алгоритм Вагнера-Фишера для поиска редакционного расстояния при произвольной стоимости операций. Также была добавлена возможность переопределять стоимость выполнения операций замены или удаления для специальных символов.

ПРИЛОЖЕНИЕ 1

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
def vagner_fisher_alg(str1, str2, replace_cost, insert_cost,
delete_cost, rep_sym, rep_cost_new, del_sym, del_cost_new):
    N = len(str1)
    M = len(str2)
    table = [[0] * (M + 1) for _ in range(N + 1)]

    def get_replace_cost(char1, char2):
        if char1 == char2:
            return 0
        elif char2 == rep_sym:
            return rep_cost_new
        return replace_cost

    def get_del_cost(char1):
        if char1 == del_sym:
            return del_cost_new
        return delete_cost

    for i in range(1, N + 1):
        table[i][0] = table[i - 1][0] + get_del_cost(str1[i - 1])
    for j in range(1, M + 1):
        table[0][j] = table[0][j - 1] + insert_cost
    for i in range(1, N + 1):
        for j in range(1, M + 1):
            table[i][j] = min(table[i - 1][j] + get_del_cost(str1[i -
1]), table[i][j - 1] + insert_cost, table[i - 1][j - 1] +
get_replace_cost(str1[i - 1], str2[j - 1]))
    for q in table:
        print(q)
    return table[N][M]

rep_cost, ins_cost, del_cost = list(map(int, input().split(' ')))
str1 = input()
str2 = input()
rep_sym,  del_sym = list(input().split(' '))
rep_cost_new, del_cost_new = list(map(int, input().split(' ')))
print(vagner_fisher_alg(str1, str2, rep_cost, ins_cost, del_cost,
rep_sym, rep_cost_new, del_sym, del_cost_new))
```